TESTING ANALYSIS DOCUMENTATION

SunAI EyeVen

AI-Powered Vendor Onboarding and Compliance Orchestration System

Document Control

System Under Test:	SunAI EyeVen – AI-Powered Vendor Onboarding and Compliance Orchestration System
Repo:	https://github.com/Lyann444/EyeVen
Project Board URL	https://trello.com/invite/b/69ed04bedaefafec12361e3e/ATTI3bad6c4820b3dedad377b70ff81e50f5ED3F54E2/eyeven-umhackathon
Tech Stack:	React, TypeScript, Material UI, Chart.js, Python Flask, MySQL, PaddleOCR, Z AI GLM, Docker, Vercel

Objective

This Testing Analysis Documentation defines the test strategy, testing risks, environment, release thresholds, and test cases for EyeVen. EyeVen transforms unstructured vendor onboarding submissions into structured, verifiable, risk-aware onboarding cases.

1. Scope and Requirements Traceability

In scope: vendor case management, document upload and extraction, intake classification, form structuring, field alignment, AI verification, policy evaluation, routing decision, follow-up email draft generation, corrected upload reprocessing, security and performance validation.

Out of scope: real SSM/government API verification, payment processing, automatic email sending, enterprise procurement integration, legal certification, and large-scale production load testing.

2. Risk Assessment and Mitigation Strategy

Critical risks include OCR extraction errors, hallucinated GLM output, mismatched vendor evidence, wrong routing approval, missing procurement form failures, GLM API failure, duplicate corrected uploads, inconsistent workflow state, and secret exposure. Mitigations include schema

validation, evidence-based prompting, confidence scores, manual review fallback, append-only run history, reprocessing runs, and repository secret scanning.

3. Test Environment and Execution Strategy

Frontend: React + TypeScript using npm start. Backend: Python Flask using python run.py. Database: MySQL 8.0. OCR: PaddleOCR. AI: Z AI GLM. API testing: Postman. Browser: Chrome. Deployment testing: Docker Compose and optional Vercel frontend.

Unit tests cover CaseService, form field seeding, file handling, OCR service, parser service, GLM wrapper, and routing state mapping. Integration tests cover case creation to workflow completion, extraction to structuring, verification to policy, policy to routing, follow-up draft generation, and reprocessing.

4. CI/CD Release Thresholds and Automation Gates

Merge gates: zero build errors, critical backend tests passing, frontend TypeScript check passing, database migration clean, API smoke tests passing, and no exposed secrets. Deployment gates: regression pass rate at least 90%, no P0/P1 bugs, AI output pass rate at least 80%, OCR usability at least 75%, no orphan workflow records, and no credential exposure.

5. Test Case Specifications

TC-01 Happy Case: create case, upload complete package, run workflow, verify low/medium risk routing.

TC-02 Missing Form: upload supporting documents only, confirm form-absent handling.

TC-03 Evidence Conflict: mismatch SSM number and confirm conflict detection.

TC-04 Unsupported File: upload corrupted/unsupported file and confirm graceful handling.

TC-05 ZIP Bundle: upload ZIP and confirm child files extracted.

TC-06 Expired ISO: verify policy risk and follow-up/manual review.

TC-07 Follow-Up Required: incomplete recoverable case generates email draft.

TC-08 Reprocessing: corrected upload updates same case without duplication.

TC-09 Performance: normal workflow completes without timeout.

TC-10 Security: no .env, API key, or service secret committed.

6. AI Output and Boundary Testing

AI-01 Complete form extraction: map values to canonical fields, no hallucination.

AI-02 Conflict detection: detect mismatched bank or SSM evidence.

AI-03 High-risk policy: classify legal disputes, missing signature, expired ISO appropriately.

AI-04 Routing: missing bank proof should require follow-up.

AI-05 Follow-up email: professional draft with only supported missing items.

Oversized input should be chunked, rejected clearly, or routed to manual review. Prompt injection inside documents must be ignored. Malformed OCR text must lead to low-confidence, missing, or unverifiable statuses rather than invented values.

7. Requirement Traceability Matrix

REQ-CM-01 to TC-01; REQ-DOC-01 to TC-01; REQ-DOC-02 to TC-01/TC-02; REQ-DOC-03 to TC-01/TC-09; REQ-DOC-05 to TC-05; REQ-FORM-03 to TC-03/AI-02; REQ-VER-01 to TC-01/AI-02; REQ-POL-01 to TC-06/AI-03; REQ-ROUTE-01 to TC-07/AI-04; REQ-FUP-01 to TC-07/AI-05; REQ-REP-01 to TC-08; REQ-NFR-01 to TC-09; REQ-NFR-02 to TC-10.

8. Defect Logging Format

Fields: Defect ID, Test Case ID, Module, Severity, Priority, Description, Steps to Reproduce, Expected Result, Actual Result, Evidence, Assigned To, Status, Retest Result.

9. Sign-Off Criteria

All critical end-to-end tests pass; no P0/P1 defects remain open; AI structured output pass rate is at least 80%; OCR extraction is usable for readable files; mismatched/missing evidence is not auto-approved; follow-up cases generate reviewable drafts; corrected uploads reprocess same case; no secrets are exposed; system runs locally with documented setup.

Repository Note

The exact repo name initial-eyeven-project was not found through the connected GitHub list. The accessible EyeVen codebase used for this documentation is Lyann444/EyeVen.